

FIRESQUE 裝備配發與 AI 測量系統

系統部署與轉移手冊 (SOP)

本系統採前後端分離架構，結合了 **Cloudflare Worker (D1 雲端資料庫)** 與 **Google 雲端生態系 (Google 試算表、雲端硬碟、表單與 Apps Script)**。兩者之間透過 API 進行雙向同步，能同時兼顧 Cloudflare D1 資料庫的高效讀取與 Google Sheets 試算表的直觀行政管理。

本手冊提供從零開始設定本系統完整環境（或轉移至全新帳號）的標準作業流程。

系統架構簡介

在開始部署前，請先理解系統各組件之間的運作邏輯：

1. 前端靜態網頁 (GitHub Pages 等)：

- 包含主頁面 `index.html` 以及 `AutoUnitUpdater.html`。
- 使用者瀏覽前端網頁進行測量與標籤列印，前端的 API 請求會直接發送給 **Cloudflare Worker**。

2. Cloudflare Worker 後端 (cf-backend)：

- 作為核心 API 伺服器，負責處理前端的讀取與寫入請求。
- 搭配 **Cloudflare D1 資料庫** 提供即時的資料讀寫（如儲存量測紀錄、更新分隊箱狀態）。
- 在寫入資料時，會在背景非同步將資料發送至 **Google Apps Script** 進行同步備份。

3. Google 雲端硬碟與試算表 (Google Apps Script / GAS)：

- Google 雲端硬碟**：專門儲存使用者上傳的測量照片。
- Google 試算表**：作為備份資料庫與行政管理後台。
- Google 表單 (選用)**：若外部使用者填寫表單，資料會寫入試算表並自動同步回 Cloudflare D1 資料庫。
- Google Apps Script (GAS)**：負責接收 Cloudflare Worker 的同步寫入，並透過活動觸發器 (Triggers) 監聽試算表變動，將試算表的編輯與刪除即時同步回 Cloudflare D1。

階段一：Google 雲端環境設定 (雲端硬碟、試算表與 GAS)

請登入您的 Google 帳號，並依照以下步驟進行設定：

1. 建立照片儲存資料夾 (Google 雲端硬碟)

1. 在 Google 雲端硬碟中，建立一個新的資料夾（例如命名為 `FIRESCUE_Photos`）。
2. 在資料夾上點選右鍵 > 共用 > 將一般存取權設為「知道連結的任何人都可查看」(Anyone with the link can view) 並設為「檢視者」。

⚠ 重要提示

必須設定此共用權限，系統前端才能正常顯示上傳的照片。

3. 進入該資料夾，並複製網址中的 資料夾 ID：
 - 網址格式為：`https://drive.google.com/drive/folders/【這一段就是資料夾ID】`。請先將這串 ID 記下來。

2. 建立 Google 試算表 (資料庫)

1. 在雲端硬碟中建立一個新的 Google 試算表。
2. 根據系統結構，在該試算表中建立以下工作表 (Sheet names)，請確保名稱完全相同：
 - `系統設定`：用於存放後台管理設定與列印參數。
 - `分隊箱狀態`：用於追蹤各分隊裝箱進度與貨運單號。
 - `單位資料`：存放機關、大隊、分隊階層架構與代碼。
 - `人員名冊`：匯入要參與測量的人員清單。
 - `尺寸參數`：系統進行尺碼推薦時的對照級距。
 - `測量紀錄`：儲存所有使用者的量測數據與推薦尺碼。
 - `職稱`：下拉式選單的職稱清單。
 - `服裝尺碼`：備用的尺碼規格。
3. 請將舊系統對應工作表中的基礎資料複製貼上至新建立的工作表中（如 `尺寸參數`、`單位資料`、`職稱` 與 `人員名冊`）。

3. 部署 Google Apps Script (GAS API)

1. 在剛剛建立的試算表中，點選上方選單「擴充功能」>「Apps Script」。

2. 將專案中的 `google_apps_script.js` 程式碼完整複製，並貼上覆蓋 GAS 編輯器中的所有內容。

3. 修改關鍵參數 (GAS 程式碼內)：

- 尋找 `Utilities.newBlob` 附近 (大約第 837 行)，找到以下設定並換成您在步驟 1 取得的資料夾 ID：

```
var folderId = "【請替換為您新建的照片資料夾ID】";
```

- 尋找 `cfUrl` 相關變數 (大約在第 787, 895, 913, 1001, 1084 行)，將這些網址暫時留空，或在您完成「階段三：Cloudflare 後端部署」取得 Cloudflare Worker 網址後，再回來替換為您的 Cloudflare API 網址：

```
var cfUrl = "https://【您的Cloudflare-Worker網址】/api/";
```

4. 儲存專案 (Ctrl + S)。

5. 點選右上角「部署」>「新增部署作業」：

- 點選左側齒輪，選擇類型為：網頁應用程式 (Web App)。
- 說明：輸入部署註記 (例如：`Firescue API V1`)。
- 執行身分：選擇 我 (您的 Google 帳號)。
- 誰可以存取：選擇 所有人 (Anyone)。

安全警告

「誰可以存取」必須選擇「所有人」，否則 Cloudflare Worker 無法透過網路與您的 Apps Script 進行同步。

6. 點擊「部署」，系統會要求授權，請點擊「核准存取權」，並選擇您的 Google 帳號。若出現「Google 尚未驗證此應用程式」警告提示，請點選「進階」>「前往不安全的頁面 (點擊允許)」完成授權。

7. 部署完成後，系統會產生 網頁應用程式網址 (Web App URL)，格式如：

```
https://script.google.com/macros/s/AKfycb.../exec
```

- 請務必複製並記下此網址，這就是您的 Google Apps Script API 網址。

4. 設定可安裝的活動觸發器 (雙向同步關鍵)

由於 Google 的安全機制，簡單的 `onEdit` 函式無法發送網絡請求。我們必須手動建立「可安裝的觸發器 (Installable Triggers)」才能將試算表的變動即時推送到 Cloudflare D1。

1. 在 Apps Script 編輯器左側選單中，點選  「觸發器」 (Triggers) 圖示。

2. 點選右下角「新增觸發器」。

3. 建立 編輯時同步觸發器：

- 選擇要執行的功能：`onEditTrigger`
- 選擇要部署的項目：`主要`
- 選擇活動來源：`來自試算表`
- 選擇活動類型：`編輯時` (On edit)
- 錯誤通知設定：`每天通知我`，點擊儲存。`

}

{

{

4. 建立 變動時同步觸發器（處理整列刪除）：

- 點擊新增觸發器。
- 選擇要執行的功能：`onChangeTrigger`
- 選擇要部署的項目：`主要`
- 選擇活動來源：`來自試算表`
- 選擇活動類型：`變動時` (On change)，點擊儲存。

5. 建立 表單提交觸發器（若有使用表單）：

- 點擊新增觸發器。
- 選擇要執行的功能：`onFormSubmitTrigger`
- 選擇要部署的項目：`主要`
- 選擇活動來源：`來自試算表`
- 選擇活動類型：`提交表單時` (On form submit)，點擊儲存。

}

}

{

階段二：Google 表單設定 (選用 - 透過表單收集測量紀錄)

若您希望透過公開的 Google 表單讓人員自行填寫身材尺寸，並同步回系統，請依照以下步驟設定：

1. 建立/關聯 Google 表單：

- 在您的 Google 試算表中，點選選單「工具」 > 「建立新表單」。
- 這會在試算表中產生一個新的工作表（通常名為「表單回覆 1」），且會自動將表單關聯到試算表中。

2. 對齊欄位標頭（極重要）：

- 請點選該工作表，將其重新命名為「測量紀錄」（若原先已有「測量紀錄」，請先將舊的重新命名或移除，確保表單關聯的那個工作表名稱完全為「測量紀錄」）。
- 請確保表單所產生的各個欄位名稱，與我們系統所需的標題完全一致。系統匯入與同步時，是透過欄位名稱來識別資料的。必要的欄位標題包括：

系統建檔時間，登記日期，機關名稱，大隊/分類，單位名稱，人員識別碼，裝袋序號，姓名，性別，年齡，職稱，量測方式，照片檔名，照片連結，身高，肩寬，胸圍，腰圍，臀圍，褲內長，配發系列，長袖，短袖，長袖操作服，短袖操作服，背心，外套，救護外套內件，戰術外套，戰術褲，褲帶，戰術帽，消防靴，狀態，現場備註，後台備註

3. 開啟自動同步觸發器：

- 確保您在「階段一、步驟 4」中，已經建立了對應 `onFormSubmitTrigger` 函式的「提交表單時」觸發器。
- 當使用者送出表單時，Google Sheets 寫入新資料的同時會啟動該觸發器，並透過 POST 請求將新增的資料推送至 Cloudflare Worker 寫入 D1 資料庫，確保前台後台資料完全同步。

階段三：Cloudflare Worker & D1 資料庫部署

後端 API 伺服器與主資料庫部署於 Cloudflare 的無伺服器環境中。

1. 準備本地環境

1. 請確保您的電腦上已安裝 [Node.js](https://nodejs.org/) (<https://nodejs.org/>)。
2. 開啟終端機 (CMD 或 PowerShell)，切換至專案底下的後端資料夾：

```
cd cf-backend
```

3. 安裝所需依賴套件：

```
npm install
```

2. 登入 Cloudflare 並建立 D1 資料庫

1. 在終端機中執行登入指令，瀏覽器會自動彈出，請登入您的 Cloudflare 帳號並授權：

```
npx wrangler login
```

2. 建立名為 `firescue-db` 的 D1 資料庫：

```
npx wrangler d1 create firescue-db
```

3. 建立成功後，終端機會輸出資料庫資訊，格式如下：

```
[[d1_databases]]
binding = "DB"
database_name = "firescue-db"
database_id = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx" # 這段就是您的 database_id
```

請複製這段輸出內容。

3. 修改設定檔 `wrangler.toml`

1. 用文字編輯器打開 `cf-backend/wrangler.toml`。

2. 替換 D1 資料庫資訊：

- 將檔案中的 `database_id` 修改為您剛剛產生的資料庫 ID。

3. 修改環境變數：

- 在 `[vars]` 區塊下，將 `APPS_SCRIPT_URL` 修改為您「階段一、步驟 3」取得的 Google Apps Script API 網址：

```
[vars]
APPS_SCRIPT_URL = "https://script.google.com/macros/s/AKfycb ... /exec"
```

4. 初始化 D1 資料庫結構 (Schema)

1. 在 `cf-backend` 目錄下執行以下指令，在 Cloudflare 雲端初始化您的資料表結構（讀取並執行 `db/schema.sql`）：

```
# 初始化線上生產環境資料庫
npx wrangler d1 execute firescue-db --remote --file=db/schema.sql
```

（選用）如果您要在本機進行開發與測試，請執行本機初始化指令：

```
# 初始化本機測試環境資料庫
npx wrangler d1 execute firescue-db --local --file=db/schema.sql
```

5. 部署 Worker

1. 在終端機執行部署指令：

```
npx wrangler deploy
```

2. 部署成功後，終端機最後會顯示部署完成的對外 URL，例如：`https://firescue-cf-backend.<你的子網域>.workers.dev`
請將這個網址複製下來。

 重要提示

現在請回到「階段一、步驟 3」：

打開 Google Apps Script 編輯器，將程式碼中所有的 `cfUrl` 變數修改為您的 Cloudflare Worker 網址（網址結尾必須加上 `/api/` 或是對應路徑，例如：

`https://firescue-cf-backend.xxxx.workers.dev/api/`）。

修改完畢後，點選 Apps Script 右上角的「部署」 > 「管理部署作業」，點選筆按鈕編輯，並將版本切換為「新版本」進行更新發布。

階段四：設定與發布前端網頁 (靜態部署)

前端網頁為純靜態 HTML，可以放置在 GitHub Pages、Cloudflare Pages、Vercel 等任何靜態網頁代管空間。

1. 修改前端設定檔中的 API 網址

1. 使用文字編輯器（如 VS Code）打開前端核心檔案 `index.html`。
2. 尋找大約第 2102 行的 `DEFAULT_API_URL` 設定：

```
const DEFAULT_API_URL = "https://【請替換為您的 Cloudflare Worker 對外網址】/";
```

注意：網址最後要加上斜線 `/`。

3. 同理，開啟 `AutoUnitUpdater.html`（單位資料自動更新工具），尋找大約第 207 行的 `APPS_SCRIPT_URL`：
 - 這裡可以直接填入您的 **Cloudflare Worker** 網址（會自動轉發），或是直接填入 **Google Apps Script API** 網址。建議改為 Cloudflare Worker 網址以確保走同一通道：

```
const APPS_SCRIPT_URL = "https://【請替換為您的 Cloudflare Worker 對外網址】/";
```

4. 若有 `index_cf_test.html` 或 `index_backup.html` 等備用網頁，也請一併修改對應行數的 API 網址。

2. 部署至靜態空間 (以 GitHub Pages 為例)

1. 將專案的所有檔案（`index.html`、`AutoUnitUpdater.html`、`CsvStickerGenerator.html` 等）上傳至您的 GitHub 儲存庫（Repository）。
2. 進入該儲存庫的 **Settings > Pages**。
3. 在 **Build and deployment** 下，將 Source 設為 `Deploy from a branch`，Branch 選擇 `main`（或 `master`）的 `/root``，點擊 **Save**。
4. 稍等 1-2 分鐘後，即可取得您的前端發布網址（例如：`https://<您的GitHub帳號>.github.io/<儲存庫名稱>/`）。

3. 設定後台對外網址 (關鍵步驟)

⚠ 重要提示

本步驟非常關鍵，否則列印標籤上的 QR Code 將會指向舊的網址，導致掃描標籤時無法正常開啟後台更新狀態！

1. 用瀏覽器打開您剛剛部署好的前端網址。
2. 點選選單或左下角進入 「後台管理」（系統預設的管理員密碼為 `admin123`）。
3. 進入後台後，切換至 「系統設定」 頁籤。
4. 修改以下欄位：
 - 系統對外網址 (Base URL)：修改為您目前的「前端網頁發布網址」（例如：`https://kevinhuang305.github.io/Oqliq-/`，注意結尾要帶有 `/`）。
 - 管理員密碼：若要保障安全，請在此處將 `admin123` 修改為您自訂的強密碼。
5. 點擊 「儲存設定」。
 - 系統會自動將此設定同時寫入 Cloudflare D1 與同步至 Google Sheets 的 `系統設定` 工作表中。

部署完成驗證測試

部署完成後，請務必依序進行以下測試，確保系統雙向同步鏈運作正常：

1. 前端測量測試：

- 開啟前端網頁，填寫一份新的測量紀錄，上傳一張測試照片，並點選送出。
- 確認前端是否提示「 上傳雲端中」並成功提示完成。

2. Cloudflare D1 資料庫檢查：

- 在 `cf-backend` 目錄下執行查詢指令，確認 D1 中已存入剛才測試的人員紀錄：

```
npx wrangler d1 execute firescue-db --remote --command="SELECT * FROM records LIMIT 1;"
```

3. Google Sheets 寫入檢查：

- 開啟您的 Google 試算表，檢查 `測量紀錄` 工作表最下方是否已自動新增該筆測試資料。
- 檢查 `照片連結` 欄位，點擊網址確認是否能正常開啟在雲端硬碟中儲存的照片。

4. 試算表反向編輯測試：

- 在 Google 試算表上直接修改剛才測試資料的「姓名」或「後台備註」。
- 等待約 2 秒鐘，至前端後台重新整理，或使用 D1 指令查詢，確認資料已被同步修改。

5. 試算表反向刪除測試：

- 在 Google 試算表上選取該列，右鍵選「刪除資料列」。
- 確認 D1 資料庫中的對應袋號紀錄已被自動同步清除（此為 `onChangeTrigger` 作用）。

6. QR Code 掃描測試：

- 在前端列印一張測試標籤，掃描上面的 QR Code，確認網址指向您的新前端網址並能順利進入該袋號的掃描更新畫面。

